Ride-Hailing Demand & Smart Charging for an Electric Fleet

Advanced Analytics and Applications

Team 04

Authors:
Georgios Kesopoulos (7396865)
Anna Goitowski (7356320)
Marius Meurer (7443647)
Thanh Huyen Dao (7431143)
Contact: mmeurer9@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

List of Figures

1 Introduction

2 Problem Description

Business context and goal. A renowned German car manufacturer is establishing a ride-hailing platform operated with a fully electrified vehicle fleet, targeting the more receptive US market first. While the client masters vehicle manufacturing, it lacks the tactical and strategic know-how to operate an electric ride-hailing fleet. As the data science team of Bane & Wayne Partners, our goal is to give the client a data-driven understanding of ride-hailing demand for a representative US city in both spatial and temporal resolution, and to translate that understanding into fleet decisions on three levels. The strategic level concerns which districts to enter first. The tactical level concerns how many vehicles to deploy and at what times and locations. The operational level concerns how to charge and reposition the vehicles. Because the fleet is fully electric, operating cost also depends on how each vehicle is charged: charging at the right times keeps recharging cost low while still ensuring enough energy for the working day. This electrification angle connects the project to the course's broader focus on sustainability-oriented analytics, but in this assignment charging is treated narrowly, as a cost-minimisation question for the operator. Since no proprietary ride-hailing data exists yet, we use the City of Chicago taxi trip records (2024 onwards) as a proxy for ride-hailing demand. Taxis are a reasonable stand-in, although they differ from app-based hailing in booking behaviour and market coverage, a limitation we keep in view when drawing conclusions.

Data-mining goal. We translate this business goal into a sequence of analytics problems that together form the project. (1) *Data preparation*: turn the large, privacy-restricted raw trip records into clean, spatially and temporally aggregated demand panels, discretising the city with a hexagonal H3 grid and enriching it with external weather data. (2) *Descriptive spatial analytics*: characterise how demand and trip patterns vary across space and time at different resolutions, and locate demand hotspots using Gaussian Mixture Models and kernel density estimation. (3) *Predictive analytics*: frame demand as a supervised regression of the target `demand_count` per spatial unit and time bucket, conditioned on calendar, weather and location. This answers questions such as "what demand can we expect on a rainy Sunday from 10 to 11am at the outskirts of the city", rather than forecasting the next hour from the current one, and it uses Support Vector Machines and feed-forward neural networks. (4) *Prescriptive analytics*: design a smart-charging agent via reinforcement learning that minimises recharging cost while avoiding the risk of running out of energy during the working day. (5) *Synthesis*: consolidate the findings into actionable recommendations for the client, including whether the operator should rely on private or public charging infrastructure. A guiding question throughout is cost-effectiveness. We benchmark each model against simpler baselines and ask whether the added complexity, such as deep learning over a kernel SVM, is worth it for the client. Success is measured with standard regression metrics (MAE, RMSE, R^2) relative to a naive benchmark for the predictive models, and by recharging cost versus an energy-shortfall penalty for the charging agent.

3 Data description

Primary data: Chicago taxi trips. The project's core dataset is the City of Chicago *Taxi Trips (2024–)* public record, used as a proxy for ride-hailing demand. It contains roughly 14.4 million individual trips spanning January 2024 to early April 2026 (extracted May 2026), with one row per trip and 23 raw columns. Each record provides an anonymised vehicle identifier (`Taxi ID`), start and end timestamps rounded to 15-minute intervals, trip duration in seconds (`Trip Seconds`) and distance in miles (`Trip Miles`), the operating company and payment type, and a fare breakdown (`Fare`, `Tips`, `Tolls`, `Extras`, `Trip Total`). Location is recorded at three nested spatial levels: Census Tract, Community Area (Chicago's 77 administrative neighbourhoods), and pickup/dropoff centroid latitude-longitude. For privacy reasons the city suppresses the Census Tract for trips in sparsely travelled areas (missing for about 56% of pickups) and rounds timestamps. This directly shapes which fields are usable as spatial units (see Data preparation). For demand modelling, only the pickup-side time and location fields are required.

External weather data. Because weather is a strong short-term driver of mobility, the trip data is enriched with historical weather for Chicago from the Open-Meteo Historical Weather API (lat 41.85, lon -87.65), at hourly and daily resolution over the same 2024-2026 window. The variables include 2 m air temperature in F°, precipitation, rain and snowfall in inches, wind speed in mph, and the WMO weather code, plus daily sunshine duration and precipitation duration in hours. These are aligned to the trip timestamps so that weather-driven demand variation is not misattributed to the calendar.

Spatial reference. A GeoJSON of Chicago's 77 community-area boundaries provides the geometry used to map and aggregate trips spatially and to anchor the H3 hexagonal grid introduced in the next section.

4 Data preparation

4.1 Data Cleaning

Data is technically cleaned before being analyzed to ensure that all columns have consistent formats and can be processed correctly. The main steps include: (1) standardizing column names; (2) converting timestamps into datetime format; (3) converting numeric and monetary columns into proper numeric types; and (4) checking for duplicate rows. This step ensures we have technically clean, processable data.

4.2 Missing Value Treatment

This section covers two steps: (1) investigating missing values and (2) defining the treatment strategy.

First, we examine the distribution of missing values to assess whether any patterns are present. In this dataset, missing values are not equally distributed across all spatial columns. Census Tract information has a very high share of missing values (55.65%). This is expected because the City of Chicago suppresses Census Tracts for some trips for privacy reasons. Therefore, Census Tracts are not suitable as the main spatial aggregation level. Community Area (2.79%) and centroid coordinates (2.74%) are much more complete and are therefore used as the main spatial units for the model. Crucially, the missing values for these variables are distributed uniformly across months and hours of the day, which means that excluding them does not distort the observed temporal demand pattern.

Dropoff variables have more missing values than pickup variables. Since the demand model focuses on where trips start, missing dropoff values are not used as grounds for removing observations from the main demand dataset.

Missing values are not removed globally, but are handled according to the specific validation or aggregation task. For the main modeling dataset — taxi demand defined by pickup location — we remove observations with null pickup coordinates. For side analyses such as trip duration, we exclude trips where duration equals zero.

4.3 Semantic Validation and Outlier Handling

After analyzing missing values, the next step is to verify whether the available values are meaningful from a business and domain perspective. We assess the reasonableness of trip start and end times, duration, distance, the relationship between distance and price, and trip fares by examining summary statistics (mean, median, and distribution) for each variable. We also confirm that pickup coordinates fall within the geographic boundaries of Chicago.

Based on this inspection, three hard filters are applied:

- **Impossible timestamps** (0.0007%): trips where the start timestamp is later than the end timestamp are excluded as physically impossible, most likely caused by data entry errors or timestamp rounding.
- **Excessive duration** (0.08%): trips with duration above 3 hours are removed, as this exceeds what is operationally plausible for an urban taxi ride in Chicago.
- **Extreme distance** (0.008%): trips with extreme distance values are excluded, as they fall outside any geographically plausible range within the Chicago metro area and are likely caused by GPS errors or data anomalies.

Together, these filters remove less than 0.1% of total trips, ensuring the modeling dataset reflects genuine demand without meaningfully reducing its size.

4.4 Aggregation

The cleaned trip-level data is aggregated along two dimensions: time and space.

(1) Temporal resolution — Time buckets are derived from the pickup timestamp. We evaluate four granularities: 1-hour, 4-hour, 8-hour, and daily intervals. Finer resolutions capture more operational detail but produce more volatile demand counts per cell; coarser resolutions are more stable but lose short-term signal. A 4-hour bucket strikes a practical balance — it is granular enough to distinguish morning, midday, evening, and night demand patterns, while remaining stable enough for reliable model training.

(2) Spatial resolution — We evaluate two spatial unit types: Community Areas and H3 hexagons. Community Areas are administratively defined and interpretable, but their irregular shapes and varying sizes limit spatial precision. H3 hexagons offer a uniform, standardized grid that can be tuned by resolution level (see Appendix). Resolutions range from R4 (~1,770 km^2, metro-wide) down to R8 (~0.74 km^2, a few city blocks). R6 (~36 km^2, roughly a large neighborhood) is selected as the primary spatial unit: it is fine enough to capture meaningful demand variation across Chicago's districts, while finer resolutions such as R7 or R8 would produce many near-zero cells, making the prediction target sparse and noisy.

(3) Filling zero-demand combinations — A standard groupby only produces rows where at least one trip occurred, leaving (location, time) pairs with zero pickups absent from the dataset. These implicit zeros are made explicit via a Cartesian product of all cells and time buckets, so the model can learn from the absence of demand. Trip-characteristic columns are set to NaN for these filled cells.

Final decision: H3 R6 × 4-hour buckets is used as the primary resolution for all subsequent modeling. This combination provides neighborhood-level spatial granularity with a meaningful temporal structure, while keeping the dataset dense enough for robust model training.

5 Data analytics

5.1 Descriptive Analytics

City-wide demand follows a two-peak daily pattern, with the strongest peak around 5pm. Weekdays are busier than weekends, while demand also varies seasonally and weather adds a smaller secondary effect. The downtown core produces many short, high-throughput trips, whereas the airports produce fewer but longer and higher-value trips. Origin-destination flows are asymmetric and idle time is highest in low-demand periods and peripheral areas. These patterns are consistent with commuting, business activity, nightlife, and airport travel, but remain descriptive rather than causal findings.

The GMM and KDE show strong concentrations around central Chicago and the airport areas. Across resolutions, coarse spatial or daily aggregation loses operational detail, while fine hexagons and hourly bins produce many empty observations. H3 r6 x 4h provides the best compromise between operational detail and sparsity and is therefore carried into predictive modelling.

5.2 Predictive Analytics

5.2.1 Modeling dataset and feature design

This section constructs two supervised models that predict taxi trip demand per spatial unit and time bucket in spatio-temporal resolution. The canonical modeling dataset is the **H3 r6 x 4h aggregation**. The prediction target is `demand_count`, the number of trips originating in a spatial unit during a time bucket. The panel is zero-filled, so the absence of trips is represented as an observation of zero rather than a missing row; this is required for the model to learn low-demand conditions rather than only observed trips. Demand is heavily right-skewed with 12.6% zeros, so it is modelled on a `log1p` scale and predictions are inverted before scoring.

Regarding independent variables, we conduct feature engineering process with these steps:

Calendar encoding. Hour-of-day, day-of-week, and month are periodic quantities, so each is encoded as a sine/cosine pair rather than a raw integer. This preserves continuity across the period boundary (23:00 is adjacent to 00:00, December to January), which an ordinal encoding would misrepresent as maximally distant. A binary weekend indicator is added because demand shifts between commuting and leisure regimes that are not fully captured by the day-of-week cycle alone.

Spatial encoding. Each spatial unit is represented by its centroid latitude and longitude rather than by a one-hot indicator per cell. Continuous coordinates encode relative position and distance, allowing the support vector machine and neural network to share information between neighbouring units and to generalise across space — which directly supports location-conditioned queries such as demand "around the outskirts". The centroid is derived deterministically from the H3 cell identifier, so reframing the retained spatial label as coordinates introduces no information loss.

Hotspot distances. The previous section identified three dominant demand centres: the downtown core, O'Hare, and Midway airports. The great-circle (Haversine) distance from each unit to these three anchors is added as an explicit feature. These distances are deterministic transforms of the centroid coordinates and therefore add no new information in principle, but they supply a coordinate frame aligned with the observed demand structure, which finite-sample SVM and neural-network models exploit more readily than raw coordinates. Three additional columns impose a negligible dimensional cost. The anchors are identified from training data only and are not hard-coded. Distances are expressed in miles for consistency with the trip-level distance fields.

Weather data. The weather features are aggregated into 4-hour bins from weather data, aligned with the demand time buckets. We use the mean for temperature and wind speed, the sum for rain, snow, and total precipitation, and the maximum weather code to capture the most severe condition within each window.

5.2.2 Validation strategy

In terms of train/test splitting, we split by grouping by calendar day, which means a day appears entirely in training data or testing data, never both. This is because the 4-hour buckets of a given day and cell share weather, calendar values, and unobserved local shocks. A purely random split could place some of a day's buckets in training and the rest in test, letting the model exploit day-specific signal absent from the features and inflating the score.

For feature standardization and hotspot anchor creation, we fit exclusively on training data. Standardization and the target transform are wrapped in a pipeline, ensuring they are automatically refit within each cross-validation fold.

For hyperparameter tuning and model selection, the NN uses five-fold GroupKFold over the development set. The linear SVM uses three-fold cross-validation, while the computationally expensive RBF-SVM uses three-fold GroupKFold on a 120-day development subsample. For testing and model comparison, a grouped random sample of whole days (15%) is set aside as an untouched test set.

To evaluate model performance, we use three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2. MAE reports the average absolute deviation between predicted and observed demand in the original unit (trips per spatio-temporal bin), making it directly interpretable for non-technical stakeholders. RMSE penalizes larger errors more heavily due to the squaring term, making it more sensitive if the model underestimates demand during a spike. R^2 indicates the proportion of variance in demand explained by the model relative to a naive mean-baseline, giving a scale-free measure of overall fit.

5.2.3 Support vector machines

First, a linear support vector regressor is fitted as a simple baseline. A small grid search selects `C = 0.1` and `epsilon = 0.2`. On the holdout set, the linear SVM achieves MAE 65.56, RMSE 203.49, and R^2 0.633. As a linear model, it cannot capture nonlinear interactions between location and time, which contributes to its larger errors during demand peaks.

Second, an RBF kernel is introduced to capture nonlinear relationships between time, location, and weather. The three-fold grouped grid search is run on a random 120-day development subsample because kernel SVMs are computationally expensive. The selected parameters are `C = 10`, `gamma = 0.1`, and `epsilon = 0.1`. The RBF-SVM improves holdout performance to MAE 30.83, RMSE 118.86, and R^2 0.875.

Its main shortfalls are computational cost and limited scalability. The selected RBF estimator is not refitted on the full development set, takes approximately 18 seconds to score the holdout, and does not outperform the historical-mean benchmark (MAE 29.14, RMSE 107.13, R^2 0.898).

5.2.4 Neural networks

We also use a feedforward neural network to predict taxi demand. Our model development process includes developing a baseline model and tuning its hyperparameters.

First, the feedforward baseline is fitted without hyperparameter tuning. It uses the same feature matrix, splits, and scaling as the SVM models. Its architecture consists of an input layer sized to the feature count, two hidden layers with 64 and 32 nodes, ReLU activations, and an output layer (see Appendix). Other settings include:

Dropout = 0.2. During training, 20% of hidden-layer activations are randomly set to zero. This regularises the network by reducing co-adaptation and dependence on individual activations.

Optimiser. Adam with lr=1e-3. By using Adam for optimiser, each parameter gets its own adaptive step size, scaled down where gradients are large or noisy, scaled up where they are small or consistent.

Early stopping. Training stops when validation loss fails to improve for 15 epochs and restores the weights from the best-performing epoch. The internal 10% validation split is drawn from the respective training fold and does not overlap with the outer validation fold or final holdout. The baseline result is shown in the Appendix.

Second, we vary the number and width of hidden layers, dropout, learning rate, and batch size. The candidate values shown in the Appendix are evaluated by manual grid search with five-fold grouped cross-validation.

Among the candidate configurations evaluated via grid search, the best-performing model was found with hidden sizes (128, 64, 32), dropout of 0.1, learning rate of 1e-3, and batch size of 512. The average `demand_count` of the dataset is 108. The MAE of the best model is 22, which is only 20% of average. It indicates that model is quite good in predicting the demand. Meanwhile, RMSE is much more larger than MAE. It may be because the model makes huge mistakes with the spatial unit that have very high demand. R2 = 93.54. It means that model can explain 93.54% of variance. In other word, it shows that model is 93.54% than the model predicts all rows as mean of demand.

To see how the variables contribute to the prediction, we analyze SHAP values (SHapley Additive exPlanations), which indicate how much each feature pushed the prediction up (+) or down (-). The SHAP value of feature i is calculated by adding feature i to every possible subset of features, measuring how much the prediction changes, and averaging that change across all subsets with appropriate weights.

The SHAP plot can be found in the Appendix {}. The most important features are related to spatial unit, especially the **distance** from the hexagon to the busy areas. Low distance to Loop makes the demand increase, which means the hexagon near the busy zones have high demand. It aligns with the business sense. The **time features** (`hour_cos`, `hour_sin`) also have the high ranks, which means time of day matters a lot. `is_weekend` (weekend = 1) slighly negative. It means that weekdays have slightly higher taxi demand overall. The **weather features** do not have much contribution to the demand, the SHAP value is nearly 0.

5.2.5 Resolution sensitivity

To assess how model performance varies with spatial and temporal resolution, we re-evaluate model performance along two dimensions: (1) spatial resolution (H3 r5/r6/r7, Community Area, Census Tract) at fixed 4h bins, and (2) temporal resolution (1h, 4h, 8h, 1d) at fixed H3 r6. We use the same set of hyperparameters and model configurations as in 4.2.3 and 4.2.4 and only refit them on each resolution's training split. The hotspot anchors and the scaler are refitted per resolution because they are data-derived.

Regarding spatial sensitivity, the granularity–sparsity trade-off is clear. From r5 to r7, the number of cells grows from 8 to 128, mean demand falls from 357 to 22, and the zero-rate climbs from 1% to 49%. R^2 weakens accordingly. Raw MAE falls because finer cells hold fewer trips; hence, cross-resolution comparisons should rely on R^2/nMAE, not raw MAE. Community Area (77 cells, R^2 0.69–0.77) sits between r6 and r7. This result shows that finer resolution gives more localized signals but degrades reliability once cells become sparse. R6 (or Community Area) is the more defensible operating resolution for fleet decisions than r7, where the low R^2 indicates limited explanatory power. However, it should be noted that the per-resolution neural network is a single refit and underperforms its tuned version.

{Note: add result table in the Appendix}

Census tract has 607 unique cells, 94.7% of buckets empty, and R^2 collapses to 0.08 (SVM) / 0.18 (NN) - essentially no better than predicting the mean. Two effects compound. First, the grid is extremely fine, so sparsity alone depresses R^2, as it already did at r7. Second, only **45.6%** of trips carry a tract and the missing ones are the low-demand ones, so even the observed demand is a biased, deflated picture. The conclusion for the client is that census tract is unsuitable as a primary unit - not because the model is weak, but because the input is both too fine and structurally incomplete in a demand-correlated way.

{Note: add result table in the Appendix}

Regarding temporal sensitivity, from 1h to 1d the zero-rate falls from 26.8% to 4.0% and R^2 rises while raw MAE grows with the bigger counts. But high R^2 at 1d is not a free win: a one-day bucket cannot answer "how many cars at 8am vs 6pm", the tactical question the client actually has. 4h is enough temporal grain for shift planning, sparse enough to stay learnable.

{Note: add result table in the Appendix}

5.2.6 SVM vs. NN comparison

On the holdout set, the tuned NN is the best model (MAE 24.9, R2 0.921), the NN baseline is essentially as good (MAE 25.8, R2 0.926), and both clearly beat the RBF-SVM (MAE 30.8, R2 0.875) and the linear SVM (MAE 65.6). Three points decide the verdict:

- **The SVM does not beat the naive benchmark.** The historical-mean lookup scores MAE **29.1 / R2 0.898** - better than the RBF-SVM on both metrics. Only the NN (MAE 24.9 / R2 0.921) adds real signal over a trivial baseline. A kernel SVM that, after a full grid search, fails to beat a lookup table is hard to justify as the deployed model.
- **Cost runs the same way.** The RBF-SVM had to be tuned on a day-subsample (kernel SVMs scale ~quadratically in samples) and takes ~**18 s** to score the holdout; the NN trains on the full development set and scores in ~**0.01 s** - roughly 1800x faster inference. The deep model is both more accurate and cheaper to serve.
- **Interpretability** is not lost: SHAP (3.6) shows time-of-day and the hotspot distances dominate - enough for a client conversation.

Therefore, deep learning is clearly worth it here: the NN is the only model that meaningfully beats the benchmark, and it is also the faster one to serve.

{Note: add result table in the Appendix}

5.3 Smart Charging Using Reinforcement Learning

5.3.1 MDP and model setup

The assignment's two-hour home-charging window is divided into eight 15-minute decisions. The state contains the slot and battery charge; actions are 0, 7, 14, or 22 kW, and 60 kWh is chosen as the battery-capacity parameter. Energy demand is modelled as `max(0, D)`, where `D` is normal with mean 21.4 kWh and standard deviation 14.1 kWh. These parameters are derived from pooled taxi-day distances using an assumed conversion of 0.40 kWh/mile. Slot cost is `alpha_t * (exp(0.1 * power) - 1)`. A shortfall incurs 20 plus 9.54 per missing kWh. The first day starts at 5 kWh; as an extension to the single-day setup, later days start with the preceding day's remainder.

5.3.2 Learning approach

Tabular Q-learning uses the slot and battery charge, snapped to a 0.5 kWh grid, with the four charging actions. It is trained for 200,000 simulated days using epsilon-greedy exploration; epsilon decreases from 1.0 to 0.02 and the learning rate decays. Future rewards are undiscounted within a day and discounted by 0.95 at the day boundary. Dynamic programming (DP) uses the same action set and battery grid and converges after 187 iterations. However, DP also approximates demand on an 81-point grid that does not reproduce the simulator's clipped normal distribution exactly. DP is therefore an approximate model-based reference, not proof of the exact optimum in the simulation environment.

5.3.3 Results and comparison

All policies are evaluated for 5,000 days using the same random seed and hence the same demand sequence. The DP policy has the lowest average charging-cost value, 1.0521, and the second-lowest shortfall rate, 0.52%. Q-learning reaches 1.3191 and 3.02%. Constant-medium charging reaches 1.2844 and 3.90%, so, based on the reported values, Q-learning reduces shortfall by 0.88 percentage points for 0.0347 additional charging cost. Maximum charging has the lowest shortfall rate, 0.46%, but costs 3.3737; the cheapest-slots heuristic reaches 2.1106 and 1.44%. Thus, Q-learning improves reliability over constant-medium charging but does not dominate all baselines. Because the output omits penalty-inclusive return, it supports a cost-reliability comparison rather than a complete ranking by the training objective.

5.3.4 Implications and limitations

The sensitivity analysis re-solves DP for each setting and evaluates each resulting policy over 2,000 days. Increasing demand's standard deviation from 5 to 20 kWh raises average charging cost from 0.8056 to 1.3114 and shortfall from 0.10% to 3.60%. Raising both penalty components from 0.5 to 4 times their base values increases charging cost from 1.0460 to 1.1248 and reduces shortfall from 0.45% to 0.25%. Tested battery capacities of 40, 60, and 80 kWh produce observed shortfall rates of 9.20%, 0.15%, and 0.00%, respectively. These are model outcomes, not dollar costs or real-world forecasts. They depend on the chosen penalty, assumed energy conversion, clipped normal demand, stylised prices, fixed charging window, and omission of charging losses, public charging, battery degradation, and fleet interactions.

6 Discussion & Outlook

6.1 Implications for the fleet operator

For the client, the main implication is that market entry should start with a focused operating area rather than immediate city-wide coverage. The cleaned Chicago taxi data, enriched with weather data and aggregated to H3 r6 x 4h, shows that demand is strongly structured by time and location. Central areas and airport-related zones require different operating logic: dense downtown areas support frequent short trips and high turnover, while airport trips are longer, more range-sensitive, and operationally more demanding. Since taxis are only a proxy for ride-hailing demand, these findings should not be transferred blindly to another market, but the pipeline can be reused for the actual target city.

Strategically, the operator should first cover persistent demand centres and the corridors between them. Tactically, the tuned neural network is the preferred demand model because it performs best against the historical-mean benchmark, while H3 r6 x 4h offers a practical compromise between operational detail and sparsity. However, the prediction target is trip demand, not fleet size. Fleet requirements still depend on trip duration, idle time, vehicle productivity, target service levels, repositioning, and charging downtime.

6.2 Further analyses using the existing data

The most important next step is to translate predicted demand into vehicle requirements. The available taxi data already contains trip duration, idle time, and taxi identifiers, which could be used to estimate how many trips one active vehicle can realistically serve per area and time bucket. Combining this productivity estimate with predicted demand would provide a first fleet-sizing model, including reserve capacity for peaks and prediction errors.

A second extension is a simple fleet simulation. Origin-destination flows can be used to model where vehicles accumulate, where shortages occur, and how much empty repositioning is needed. This would connect the descriptive flow analysis with the predictive demand model and the smart-charging component. Instead of only asking where demand occurs, the operator could evaluate whether enough vehicles are available in the right places at the right times.

Further model validation should focus on robustness. Rolling time-based validation, prediction intervals, and residual analysis would show whether the model remains reliable across seasons, weather conditions, and unusual demand periods. This matters because operational planning depends not only on average performance, but also on how badly the model fails in peak situations.

6.3 Additional external data sources

Several external data sources could improve the analysis. Point-of-Interest data from OpenStreetMap would be especially useful, because offices, airports, transit stations, hotels, residential areas, and entertainment venues help explain why demand differs between spatial cells. Event calendars, public holidays, traffic speeds, road closures, and public-transport disruptions could further improve demand and travel-time predictions.

For the charging part, the current model should be extended with real vehicle and infrastructure data. Relevant sources include vehicle telematics, battery state-of-charge records, energy consumption by trip type, home or depot availability, electricity tariffs, grid constraints, and public charger data. Public charger data should include location, power level, price, utilisation, waiting times, and outages. Without this information, the charging model should be interpreted as a controlled proof of concept rather than a final infrastructure decision.

6.4 Charging infrastructure recommendation

The results support a hybrid charging strategy. Private charging should form the operational backbone because it offers more control over timing, availability, and cost. The smart-charging model shows that scheduled charging can maintain high reliability without always using maximum power. This is especially valuable for vehicles with predictable home or depot charging windows.

Public charging should complement private charging rather than replace it. It is useful as a reserve option for unexpectedly high energy demand, low starting charge, vehicles operating far away from private chargers, or peak-demand days where normal charging windows are insufficient. However, public charging introduces uncertainty through detours, waiting times, charger availability, and price variation, none of which are fully captured in the current model.

Therefore, the client should not decide immediately between fully private or fully public charging. A sensible next step is a pilot in the target market. The pilot should collect local ride-hailing demand, vehicle utilisation, energy consumption, charging behaviour, and public charging reliability data. Based on this evidence, the operator can size a private charging backbone and define how much public charging capacity is needed as flexible backup.

7 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like "R for Data Science" (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

8 Conclusion

The conclusion should summarize your findings and suggest future work.

9 Appendix

9.1 H3 hexagon area corresponding to specific resolution

Resolution	Avg Area	Rough Scale
R4	~1,770 km^2	Large city / metro region
R5	~253 km^2	City district
R6	~36 km^2	Large neighborhood
R7	~5.2 km^2	Neighborhood
R8	~0.74 km^2	A few city blocks

Need to add: percentage of zero cells for each combination of temporal, spatial unit

9.2 Baseline Neural Network Architecture

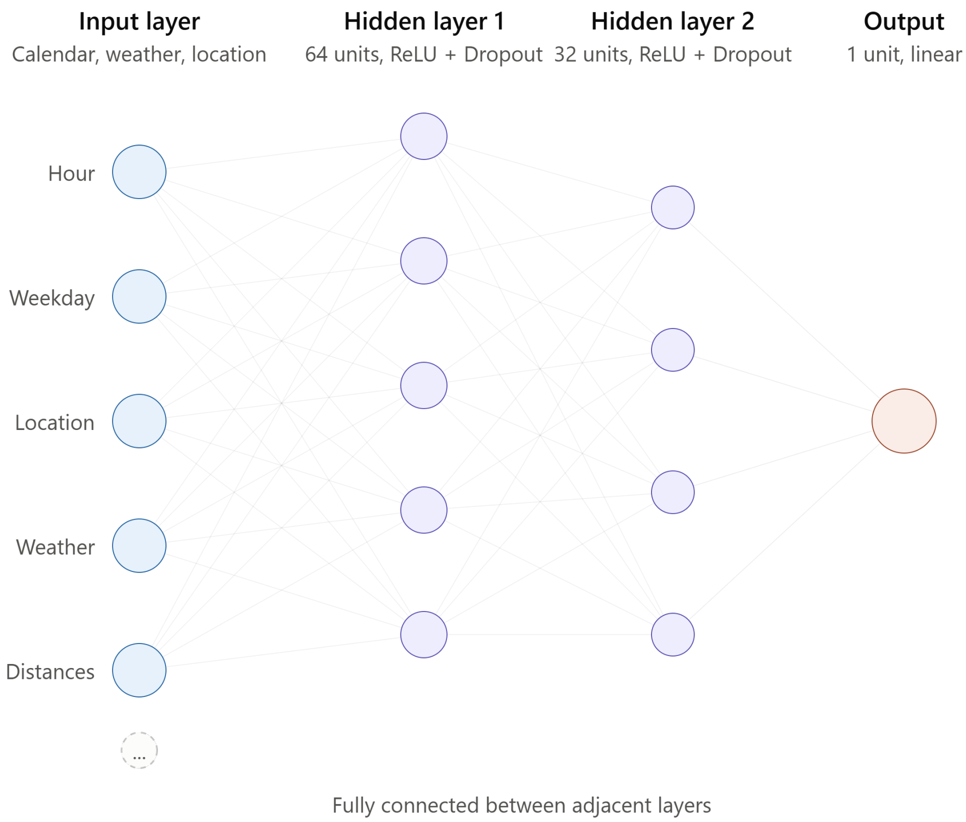


Figure 1: Model architecture

9.3 candidate values of hyperparameters for tuning Neural Network model

Parameter	Candidate Values
`hidden_sizes`	(64, 32), (128, 64, 32)
`dropout`	0.1, 0.2
`lr`	1e-3, 5e-4
`batch_size`	256, 512

9.4 Baseline Neural Network model result

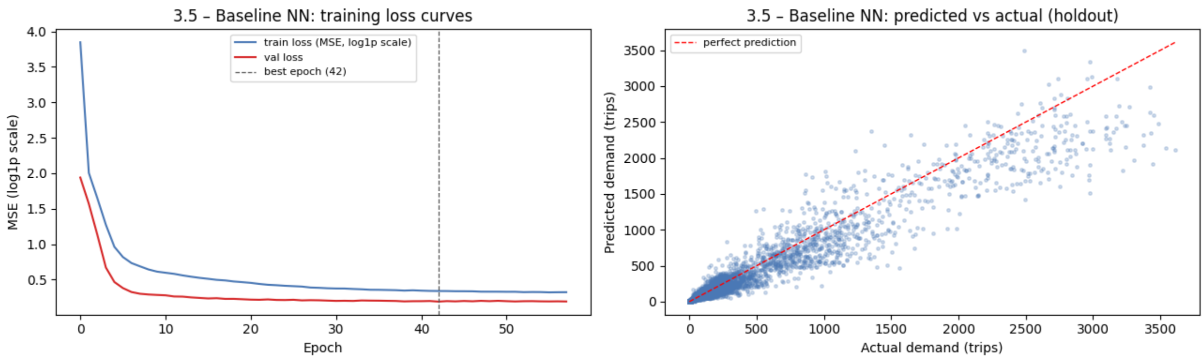
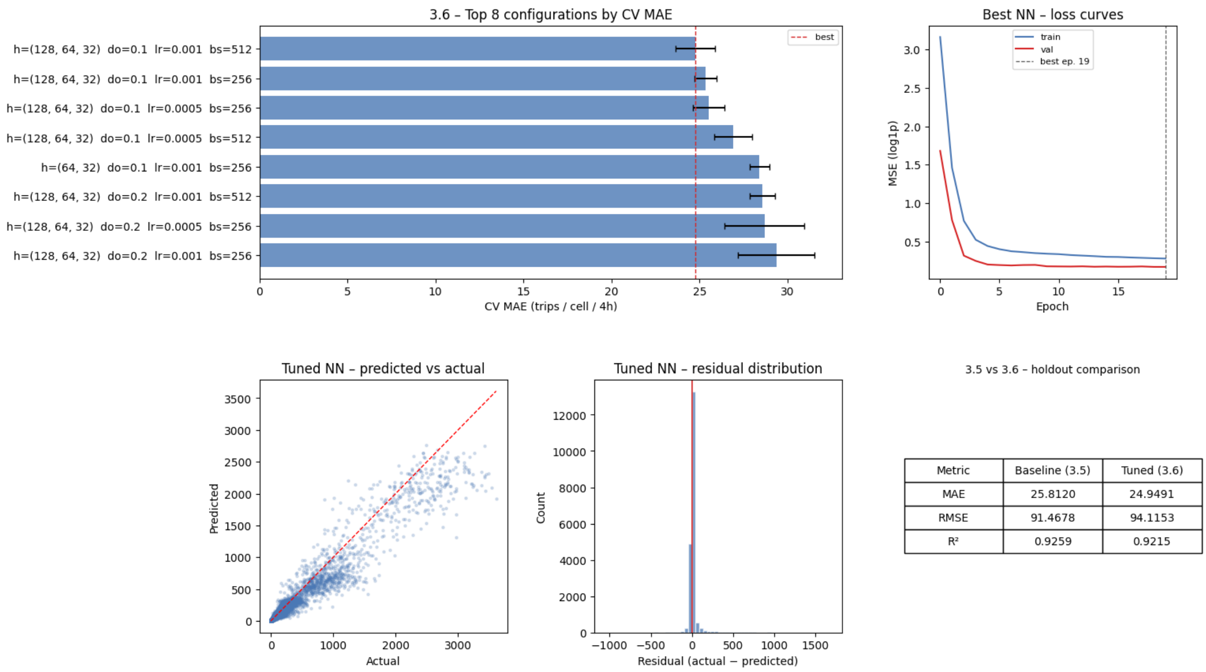


Figure 2: Model result - Neural network baseline model

9.5 Best Neural Network model



Metric	Baseline (3.5)	Tuned (3.6)
MAE	25.8120	24.9491
RMSE	91.4678	94.1153
R²	0.9259	0.9215

Figure 3: Model result - Neural network tuning model

9.6 Feature importance - SHAP - of best Neural Network model

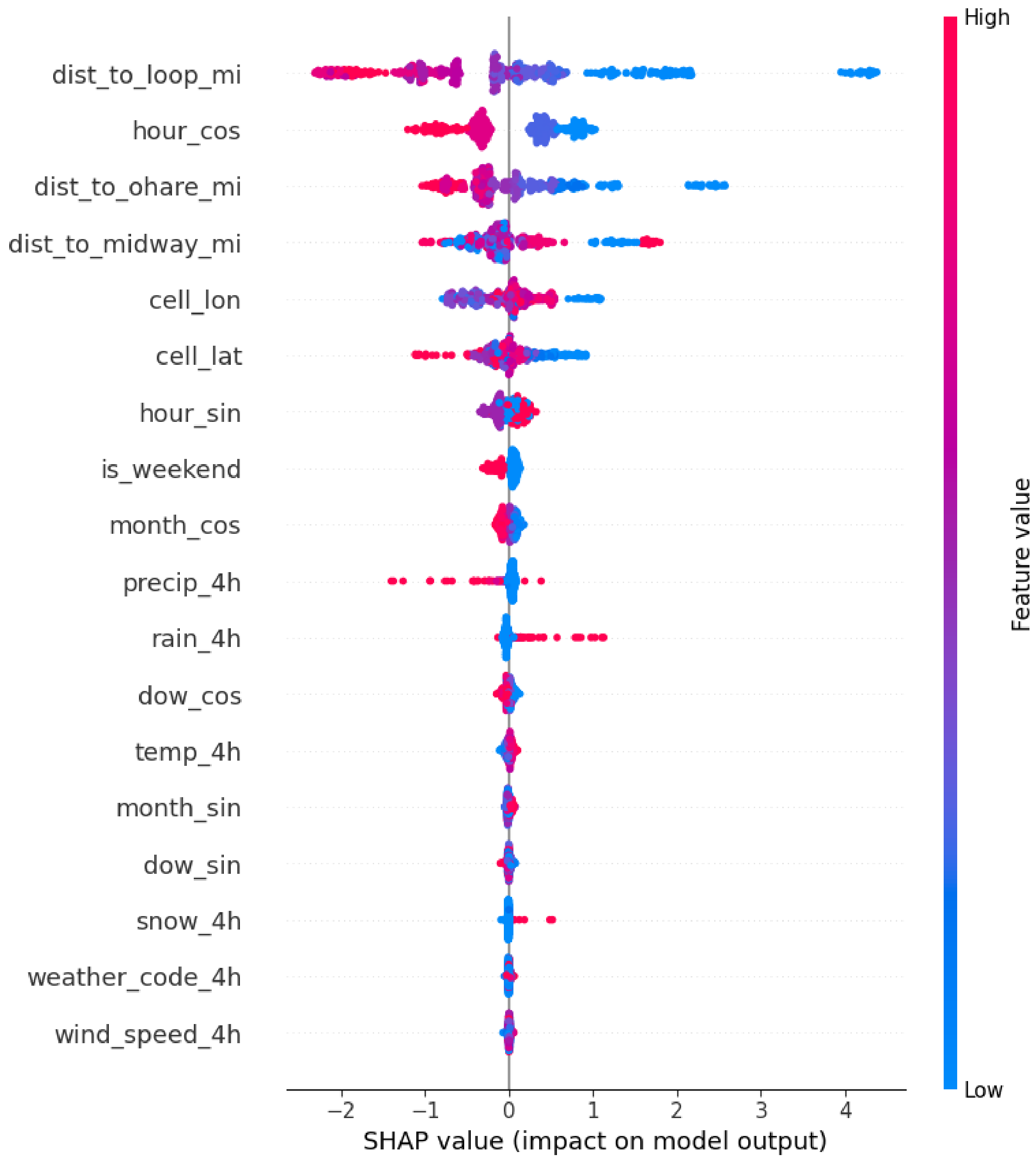


Figure 4: SHAP feature importance

References

Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. https://quarto.org

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.